

Introducción a VHDL

Matías Javier Oliva
Pablo García

Lenguajes de descripción de Hardware (HDL)

- En un principio los sistemas digitales se describían con su circuito esquemático.
- Esto trae varios problemas:
 - Se torna dificultoso describir circuitos complejos.
 - La implementación varía según la tecnología empleada.
- Es más práctico describir el hardware con algún método abstracto y después sintetizarlo para obtener el circuito equivalente.
- Surgen los HDL.

**¡No perder de vista que lo que se
escribe en HDL define una
arquitectura que debe sintetizarse!**



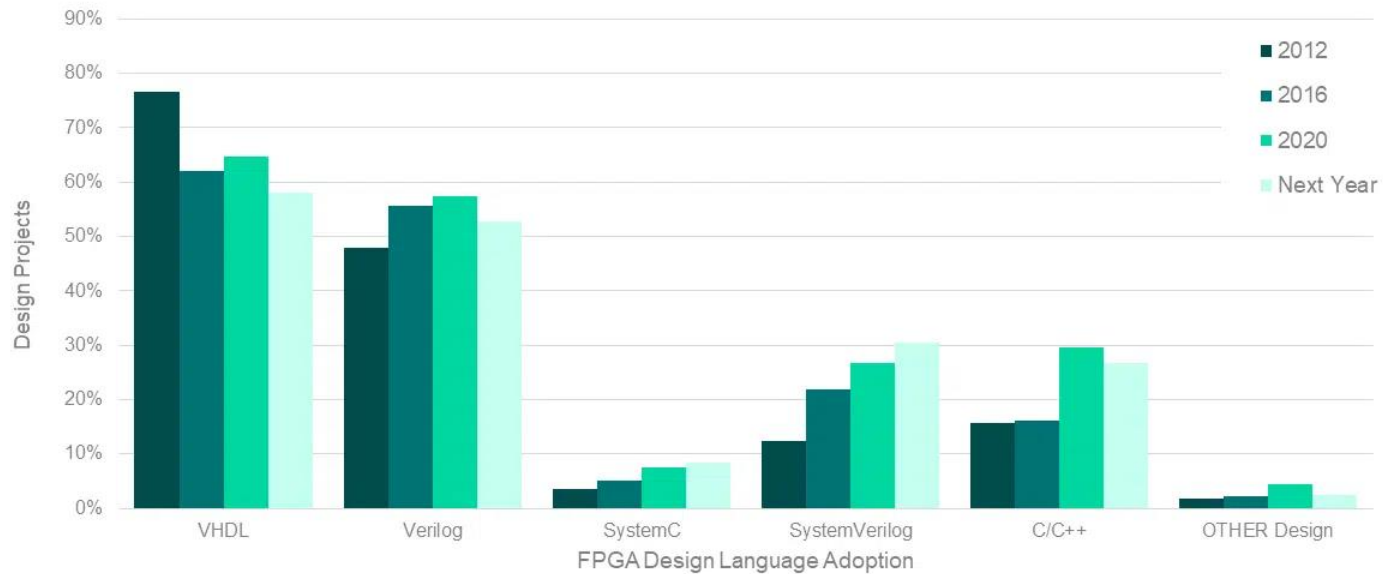
Lenguajes de descripción de Hardware (HDL)

- Existen varios lenguajes de descripción de Hardware.
- Los más utilizados son:
 - Verilog.
 - System Verilog.
 - VHDL (***V**ery **H**igh **S**peed **I**ntegrated **C**ircuit (**VHSIC**) **H**ardware **D**escription **L**anguage*).
- Otra alternativa es escribir la lógica en lenguajes de alto nivel, como C y después inferir la lógica con herramientas de High Level Syntesis (HLS), pero no es muy recomendable en su estado actual.



Lenguajes de descripción de Hardware (HDL)

FPGA Design Language Adoption Next Twelve Months



Source: Wilson Research Group and Mentor, A Siemens Business, 2020 Functional Verification Study

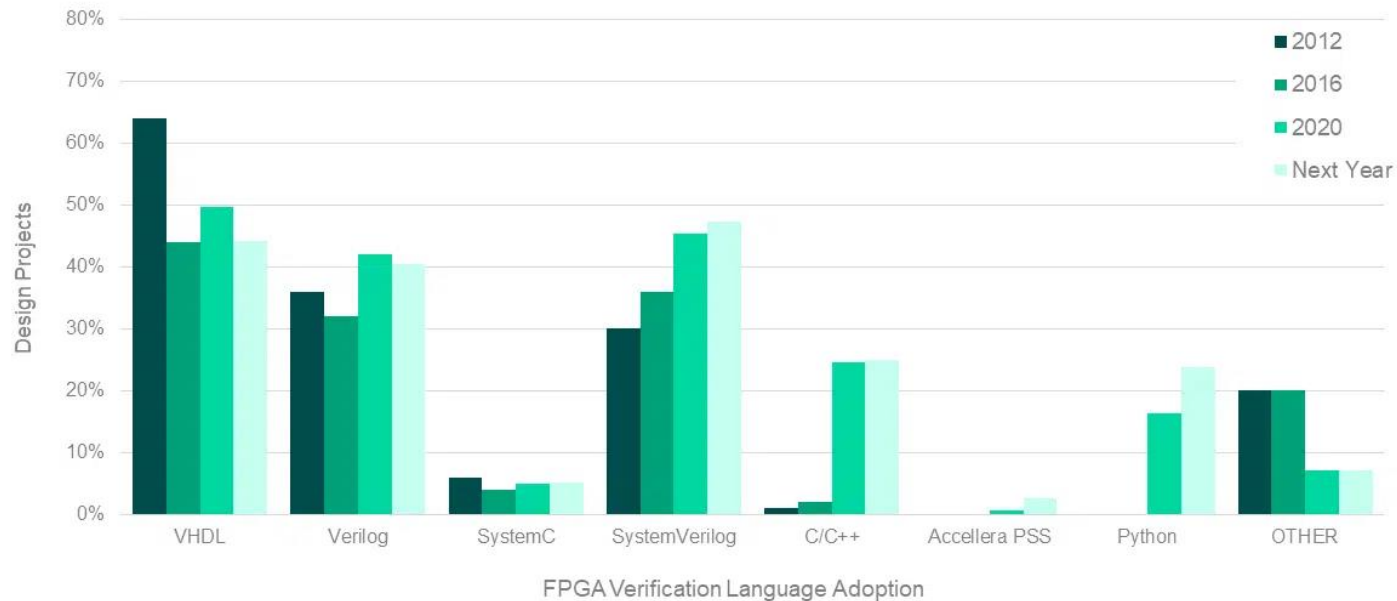
Page 1 © Siemens 2020 | 2020-10-15 | Siemens Digital Industries Software | Where today meets tomorrow.

** Multiple answers possible

SIEMENS

Lenguajes de descripción de Hardware (HDL)

FPGA Verification Language Adoption Next Twelve Months



Source: Wilson Research Group and Mentor, A Siemens Business, 2020 Functional Verification Study

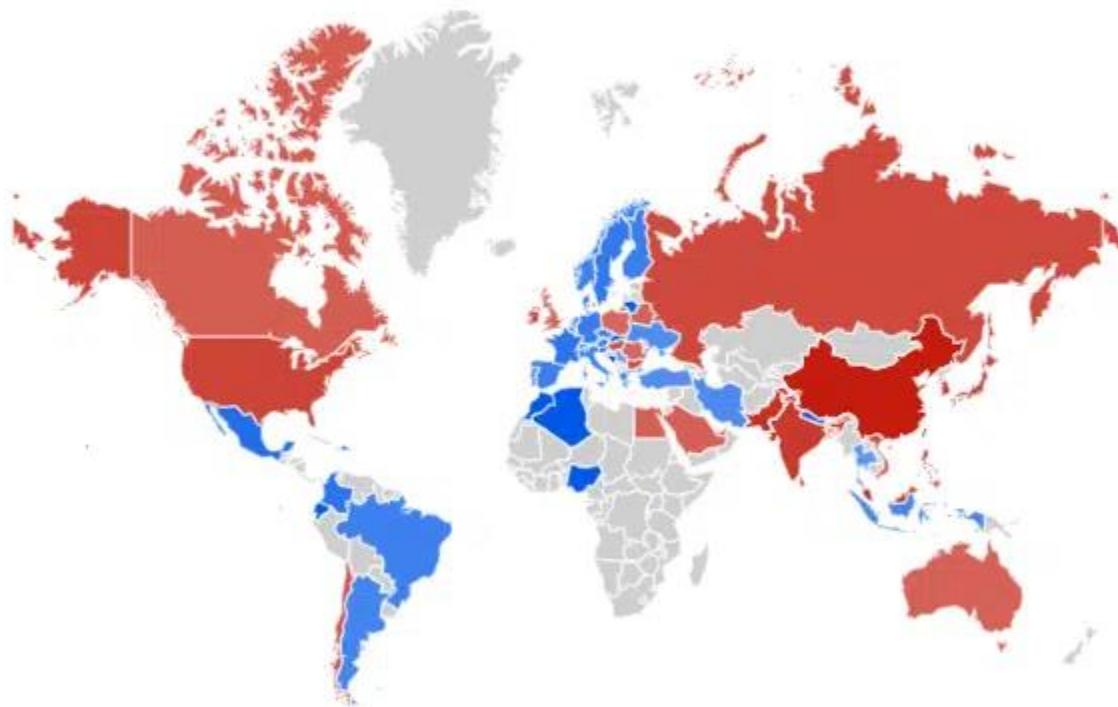
Page 2 © Siemens 2020 | 2020-10-15 | Siemens Digital Industries Software | Where today meets tomorrow.

** Multiple answers possible

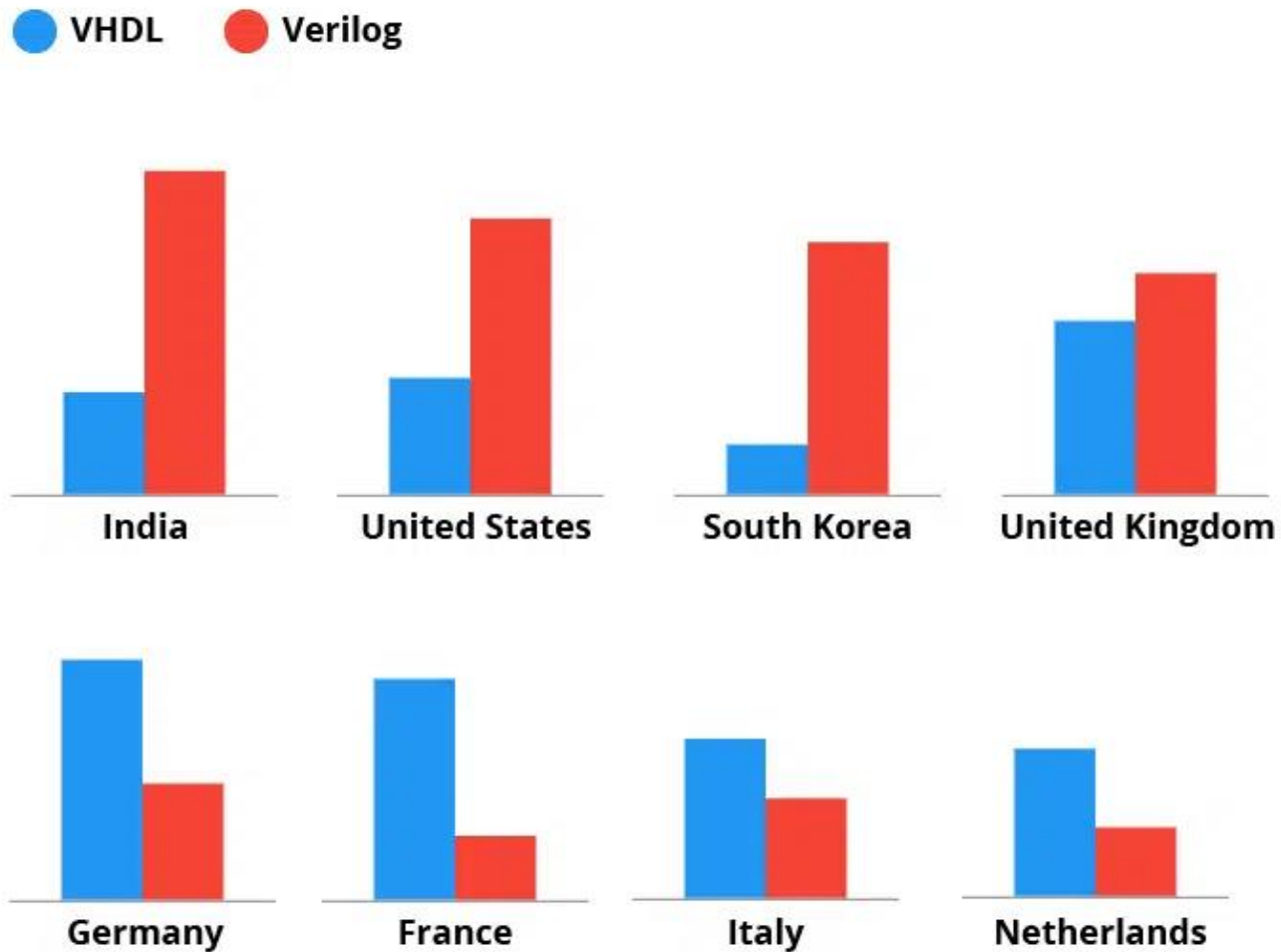
SIEMENS

Lenguajes de descripción de Hardware (HDL)

● vhdl ● verilog



Lenguajes de descripción de Hardware (HDL)



Lenguajes de descripción de Hardware (HDL)

- En este curso nos vamos a centrar en VHDL, desarrollado en la década del 80. Se supondrá que los participantes tienen un conocimiento previo de las estructuras de hardware subyacentes, como:
 - Multiplexores
 - Barrel shifters
 - Flip-flops
 - Registros de desplazamiento
 - Contadores
 - Circuitos aritméticos
 - Máquinas de estado

VHDL: Reglas generales

- VHDL es un lenguaje de descripción de hardware concebido originalmente para modelado y simulación, no para síntesis.
- Esto implica que no todo lo que uno escriba en VHDL es sintetizable (no todo tiene una correspondencia directa con hardware).
- Por otro lado, una misma funcionalidad puede describirse de muchas maneras.
- VHDL no distingue entre minúsculas y mayúsculas.
- Los comentarios se indican con --.

VHDL: Palabras reservadas

abs	disconnect	is	out	sli
access	downto	label	package	sra
after	else	library	port	srl
alias	elsif	linkage	postponed	subtype
all	end	literal	procedure	then
and	entity	loop	process	to
architecture	exit	map	pure	transport
array	file	mod	range	type
assert	for	nand	record	unaffected
attribute	function	new	register	units
begin	generate	next	reject	until
block	generic	nor	return	use
body	group	not	rol	variable
buffer	guarded	null	ror	wait
bus	if	of	select	when
case	impure	on	severity	while
component	in	open	signal	with
configuration	inertial	or	shared	xnor
constant	inout	others	sla	xor

Nota: algunas palabras reservadas de Verilog como input y output pueden generar problemas con las herramientas de simulación!.

Estructura general de un archivo VHDL

- VHDL por si solo tiene pocos tipos de datos (boolean, bit, integer), y pocos operadores entre ellos.
- Como regla general siempre se importa la librería estándar:

```
library ieee;  
use ieee.std_logic_1164.all;
```

- Sentencia library: biblioteca estándar IEEE, que contiene muchas definiciones útiles para diseño digital.
- Sentencia use: importa todo lo definido en el paquete std_logic_1164, donde están los tipos std_logic, std_logic_vector y las operaciones lógicas asociadas (como and, or, not, etc.).

Estructura general de un archivo VHDL

- Entidad: Describe el contenido del archivo como si fuera una “caja negra”.

```
entity <entity_name> is
    port
    (
        -- Input ports
        <name>      : in  <type>;
        <name>      : in  <type> := <default_value>;

        -- Output ports
        <name>      : out <type>;
        <name>      : out <type> := <default_value>
    );
end <entity_name>;
```

Nota: Por simplicidad se omiten acá las señales inout y los generic

Tipos de datos: Estándar de VHDL

Tipo de dato	Descripción
bit	Valores lógicos binarios '0' y '1'
bit_vector	Arreglo de bits
integer	Números enteros de 32 bits (su rango puede limitarse)
natural	Números enteros ≥ 0
positive	Números enteros > 0
boolean	Valores lógicos (true o false)
real	Números reales (no sintetizable)

En general no vamos a usar estos tipos de datos sino los que están definidos en `std_logic_1164`

Tipos de datos: std_logic

- Definido en std_logic_1164 es el tipo de dato que vamos a utilizar en casi todos los diseños de este curso.
- Tiene 8 valores posibles:

'X'	Desconocido
'0'	Nivel lógico bajo
'1'	Nivel lógico alto
'Z'	Alta impedancia
'W'	Desconocido débil
'L'	Nivel bajo débil
'H'	Nivel alto débil
'-'	No importa

- El tipo de dato std_logic_vector es una agrupación de señales de tipo std_logic.
- En esta librería también está el tipo std_ulogic que agrega el valor 'U' (sin inicializar) a la lista de posibilidades.
- También se puede definir std_ulogic_vector.

Tipos de datos: numeric_std

- Para hacer operaciones aritméticas no se pueden utilizar directamente las funcionalidades de std_logic_1164.
- Para hacer operaciones directamente se usan los tipos signed y unsigned, definidos en numeric_std:

Tipo de dato	Descripción	Operaciones
Unsigned	Vector binario sin signo ≥ 0	+, -, *, /, comparaciones
Signed	Vector binario con signo en complemento A2	+, -, *, /, comparaciones

- De todas maneras en este curso vamos a priorizar tener control total sobre la aritmética, describiendo el hardware a implementar en lugar de dejárselo al sintetizador

Estructura general de un archivo VHDL

- Arquitectura: Describe el funcionamiento del sistema.

```
architecture <arch_name> of <entity_name> is
    -- Declaraciones (extra, aparte de entradas y salidas)
begin
    -- Funcionamiento del sistema
end <arch_name>;
```

- Se pone después de la entidad.
- Antes del begin se declaran señales internas del diseño, y componentes extra si los hubiera.
- Entre begin y end se describe la funcionalidad.

Declaraciones de datos

```
data_object <name> : <type> := <default_value>;
```

Tipo de objeto	Propiedad	Declaración
constant	No cambia su valor a lo largo del programa	En general en parte declarativa de la arquitectura
variable	Local. Útil para cálculos temporales y lógica interna	Dentro de un proceso
signal	Global o local	En general en parte declarativa de la arquitectura

- La diferencia más importante entre variable y signal es que, si se usan en un proceso, variable toma inmediatamente el valor deseado, signal toma el valor al final del proceso

Tipos de datos: std_logic

- Instanciación de std_logic y std_logic_vector:

```
signal F : std_logic := '0';
```

```
signal G : std_logic_vector (3 downto 0) := "1000";
```

```
signal H : std_logic_vector (0 to 3) := "1000";
```

- Al declarar std_logic_vector se pueden tomar dos convenciones distintas, de izquierda a derecha o de derecha a izquierda.
- Es importante elegir una y mantenerla.
- Es recomendable de izquierda a derecha, con la palabra clave “downto”.

Literales

Tipo de Dato	Ejemplo de Literal	Descripción / Observaciones
std_logic	'0', '1', 'Z', 'U'	Literal para tipo std_logic. Admite '0', '1', 'Z', 'X', 'U', 'W', 'L', 'H', '-'. -
std_logic_vector	"1010", "ZZ01"	Vector de bits lógicos, entre comillas dobles.
bit	'0', '1'	Solo '0' o '1'. Usado en tipo bit.
bit_vector	"1010" o X"AC"	Vector de bits. Solo '0' y '1'. No admite 'Z', 'X', etc.
boolean	true, false	Valores lógicos.
integer	42, -10	Números enteros, positivos o negativos.
Binario (enteros)	B"1010", 2#1010#	Literal en binario (prefijo B o base 2).
Octal (enteros)	O"17", 8#17#	Literal en octal.
Hexadecimal (enteros)	X"1F", 16#1F#	Literal en hexadecimal.
real	3.14, -2.0, 1.0E3	Números reales, con o sin notación exponencial (E).
character	'A', '%', '9'	Cualquier carácter ASCII entre comillas simples.
string	"Hola", "1234"	Cadenas de caracteres entre comillas dobles.
Tiempo (time)	10 ns, 5 ms	Unidades: fs, ps, ns, us, ms, sec, min, hr.
null	null	Literal nulo, para punteros o tipos access.

Operadores en VHDL

Operadores de asignación

Operador	Nombre	Tipo de uso	Ejemplo de uso
<code><=</code>	Asignación a señal	Asigna un valor a una señal	<code>Y <= A and B;</code>
<code>:=</code>	Asignación a variable	Asigna un valor a una variable. También se usa para inicialización (no sintetizable muchas veces)	<code>temp := A + 1;</code>
<code>=></code>	Asociador de elementos	Mapea nombres a valores (puertos, arreglos, generics)	<code>port map (A => X, B => Y);</code>
<code>&</code>	Concatenación	Une dos vectores o elementos	<code>Z <= '0' & A;</code> <code>Z <= A & B;</code>

¡En realidad no es de asignación pero no entraba en otro grupo!

Operadores lógicos

Operador	Nombre	Tipo de operación	Usable con	Ejemplo
and	Conjunción	Lógica bit a bit	boolean, bit, std_logic, vectores	Y <= A and B;
or	Disyunción	Lógica bit a bit	Idem arriba	Y <= A or B;
nand	Negación de and	Lógica bit a bit	Idem arriba	Y <= A nand B;
nor	Negación de or	Lógica bit a bit	Idem arriba	Y <= A nor B;
xor	Exclusiva	Lógica bit a bit	Idem arriba	Y <= A xor B;
xnor	Igualdad lógica	Lógica bit a bit	Idem arriba	Y <= A xnor B;
not	Negación	Unaria (1 operando)	Idem arriba	Y <= not A;

Operadores de desplazamiento lógico

Operador	Nombre	Descripción	Tipo requerido	Ejemplo
sll	Shift Left Logical	Desplaza bits a la izquierda, llena con ceros	std_logic_vector, unsigned, signed	Y <= A sll 1;
srl	Shift Right Logical	Desplaza bits a la derecha, llena con ceros	Idem	Y <= A srl 2;
sla	Shift Left Arithmetic	Como sll, pero conserva signo si es signed	Solo signed	Y <= A sla 1;
sra	Shift Right Arithmetic	Desplaza y extiende el bit de signo a la izquierda	Solo signed	Y <= A sra 1;
rol	Rotate Left	Rota bits a la izquierda (circular)	std_logic_vector, unsigned, signed	Y <= A rol 1;
ror	Rotate Right	Rota bits a la derecha (circular)	Idem	Y <= A ror 2;

Operadores relacionales

Operador	Nombre	Devuelve	Tipos compatibles	Ejemplo
=	Igual a	boolean	boolean, bit, integer, real, std_logic, unsigned, signed, etc.	if A = B then
/=	Distinto de	boolean	Idem	if A /= B then
<	Menor que	boolean	integer, real, unsigned, signed	if A < B then
<=	Menor o igual que	boolean	Idem	if A <= B then
>	Mayor que	boolean	Idem	if A > B then
>=	Mayor o igual que	boolean	Idem	if A >= B then

Operadores aritméticos

Operador	Nombre	Operación	Tipos compatibles	Ejemplo
+	Suma	Suma de dos operandos	integer, real, signed, unsigned	$Z \leq A + B;$
-	Resta	Resta de dos operandos	Idem	$Z \leq A - B;$
*	Multiplicación	Producto	Idem (aunque real y vectores no siempre se sintetizan)	$Z \leq A * B;$
/	División	División	integer, real (⚠ limitado en síntesis)	$Z \leq A / B;$
mod	Módulo	Resto de división entera	integer	$Z := A \bmod B;$
rem	Resto	Resto con signo del numerador	integer	$Z := A \bmod B;$
abs	Valor absoluto	Devuelve el valor positivo	integer, signed	$Z := \text{abs}(A);$
**	Exponente	Potencia	real (⚠ solo para simulación, no síntesis)	$Z := A ** 2.0;$

Ejemplo completo

$$Y_1 = A \bar{B} + \bar{A}B + \bar{A}\bar{B}$$

$$Y_2 = \bar{B} + \bar{A}B$$

```

library ieee;
use ieee.std_logic_1164.all;

entity Clasel is

    port
    (
        -- Input ports
        A  : in  std_logic;
        B  : in  std_logic;

        -- Output ports
        Y1 : out std_logic;
        Y2 : out std_logic
    );

end Clasel;
```

```

architecture arch of Clasel is

    signal C,D,E : std_logic;

begin

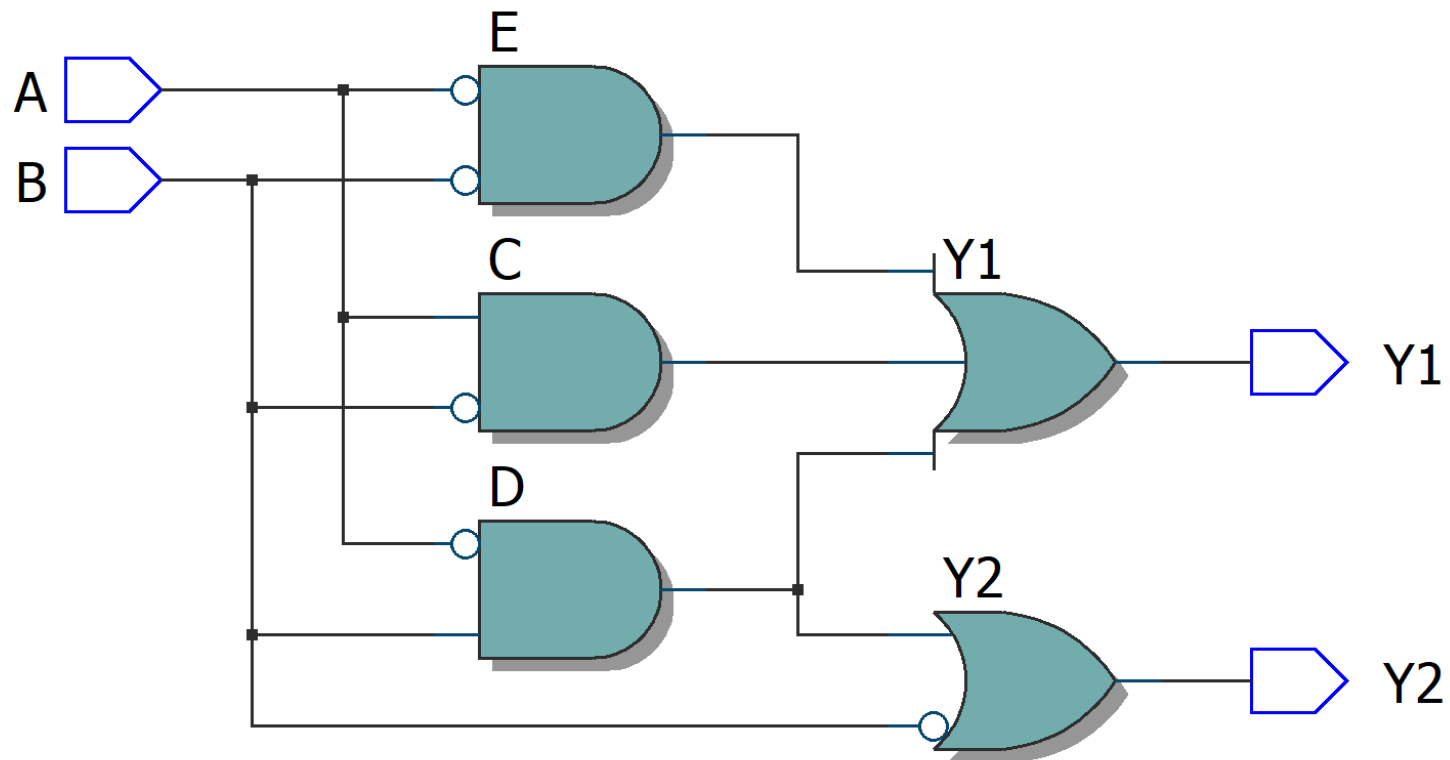
    -- Sin simplificar y con señales intermedias:
    C <= A and not(B);
    D <= not(A) and B;
    E <= not(A) and not(B);

    Y1 <= C or D or E;

    -- Simplificado
    Y2 <= not(B) or ( not(A) and B );

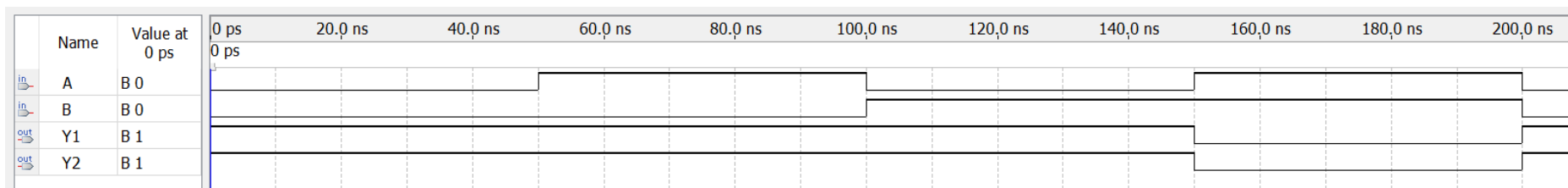
end arch;
```

Diagrama RTL (register transfer level)



Simulación

- En el diseño de HDL es clave la fase de simulación para verificar los diseños implementados.
- Existen varias maneras de simular en Intel Quartus Prime. En este curso nos vamos a centrar en dos.
 - University Program VWF (Vector Waveform File)
 - ModelSim Altera



- Por ejemplo la simulación con el University Program VWF muestra que ambas señales del ejemplo dan los mismos resultados

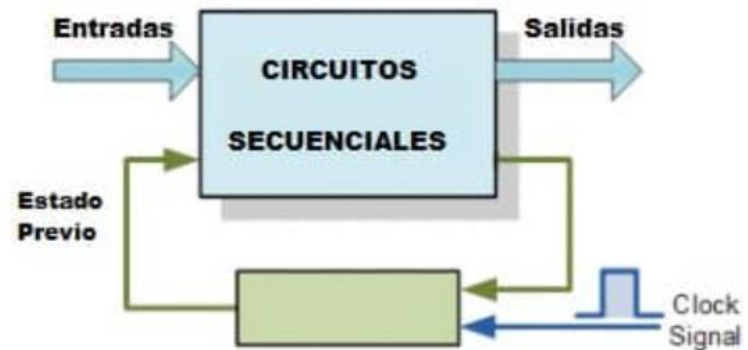
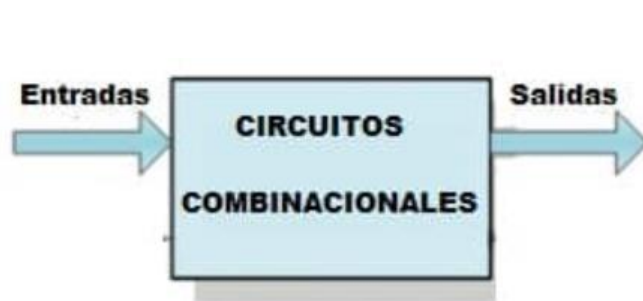
Uso de recursos

	Resource	Usage
1	▼ Total logic elements	1 / 14,400 (< 1 %)
1	-- Combinational with no register	1
2	-- Register only	0
3	-- Combinational with a register	0
2		
3	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	0
2	-- 3 input functions	0
3	-- <=2 input functions	1
4	-- Register only	0
4		
5	▼ Logic elements by mode	
1	-- normal mode	1
2	-- arithmetic mode	0
6		
7	▼ Total registers*	0 / 14,733 (0 %)
1	-- Dedicated logic registers	0 / 14,400 (0 %)
2	-- I/O registers	0 / 333 (0 %)
8		
9	Total LABs: partially or completely used	1 / 900 (< 1 %)
10	Virtual pins	0

▼ I/O pins	4 / 81 (5 %)
-- Clock pins	0 / 6 (0 %)
-- Dedicated input pins	0 / 12 (0 %)
Global signals	0
M9Ks	0 / 60 (0 %)
Total block memory bits	0 / 552,960 (0 %)
Total block memory implementation bits	0 / 552,960 (0 %)
PLLs	0 / 3 (0 %)
Global clocks	0 / 20 (0 %)
JTAGs	0 / 1 (0 %)
CRC blocks	0 / 1 (0 %)
ASMI blocks	0 / 1 (0 %)
GXB Receiver channel PCSs	0 / 2 (0 %)
GXB Receiver channel PMAs	0 / 2 (0 %)
GXB Transmitter channel PCSs	0 / 2 (0 %)
GXB Transmitter channel PMAs	0 / 2 (0 %)
Impedance control blocks	0 / 3 (0 %)
Average interconnect usage (total/H/V)	0% / 0% / 0%
Peak interconnect usage (total/H/V)	0% / 0% / 0%
Maximum fan-out	2
Highest non-global fan-out	2
Total fan-out	13
Average fan-out	0.68

Lógica combinacional vs lógica secuencial

- Las funciones lógicas se pueden dividir en dos:
 - Por un lado están aquellas que solo dependen de los valores actuales de las señales: Lógica combinacional (compuertas and, or, xor, multiplexores...).
 - Por otro lado aquellas que utilizan memoria, es decir dependen de valores actuales y pasados de las señales: Lógica secuencial. Este tipo de lógica se basa en flip-flops (En general el FFD)



Process

- Una palabra reservada clave en VHDL es el process. Este puede usarse para implementar lógica combinacional o lógica secuencial.
- En general resulta más útil para describir lógica secuencial.
- Para describir lógica combinacional también resulta útil, ya que permite implementar lógica de control “secuencial”, más parecida a lo que estamos acostumbrados del software (case, if, etc...).

Process

```
architecture test of entity1 is
begin
  <optional_label>:
    process(<sensitivity_list>) is
      -- Declaration(s)
    begin
      -- Sequential Statement(s)
    end process;
end test;
```

- La lista de sensibilidad le dice al sintetizador ante qué cambios de señales tiene que “ejecutar” lo que esta dentro del proceso.
- Se pueden declarar variables dentro del proceso.
- Si bien las instrucciones dentro del process son evaluadas en orden, el sintetizador analiza la lógica completa y genera lógica combinatorial.

Process: Ejemplo anterior

```
process(A,B)
variable F,G,H : std_logic;
begin

    if((A = '1') and (B = '0')) then
        F := '1';
    else
        F := '0';
    end if;

    if((A = '0') and (B = '1')) then
        G := '1';
    else
        G := '0';
    end if;

    if((A = '0') and (B = '0')) then
        H := '1';
    else
        H := '0';
    end if;

    if((F = '1') or (G = '1') or (H = '1')) then
        Y3 <= '1';
    else
        Y3 <= '0';
    end if;
end process;
```

- Lo que hicimos antes sin process ahora se podría hacer con process.
- Resulta un poco más parecido a programar software.
- No perder de vista que lo generado igual es combinacional.
- Este ejemplo también muestra el uso de variable (aunque podría haberse usado signal)

Bibliografía

- Orhan Gazi, *A Tutorial Introduction to VHDL Programming*. Springer, 2016. DOI: 10.1007/978-3-319-34177-0
- Brock J. LaMeres. *Introduction to logic circuits & logic design with VHDL*. Springer 2017. DOI: 10.1007/978-3-030-12489-2.
- Machado, Felipe y Borromeo, Susana y Rodriguez, Cristina. *Diseño de sistemas digitales con VHDL*. Universidad Rey Juan Carlos 2011.
- Volnei A. Pedroni. *Circuit Design with VHDL*. MIT Press, 2010. ISBN: 978-0-262-01388-2
- Documentación de Intel Quartus Prime, hojas de datos de componentes, y manuales de uso de FPGAs y SoC-FPGA disponibles en los sitios de los fabricantes.
- Tutoriales online recomendados:
 - <https://nandland.com/>
 - <https://vhdlwhiz.com/>
 - <https://www.asic-world.com/vhdl/>